CG2271 Real Time Operating Systems

2025/26 Semester 2

Final Report

Problem statement

Many elderly people in Singapore take multiple medications with different dosages and schedules, which can make timely and accurate medicine intake difficult. Hence, as an elderly patient in Singapore who takes multiple medications daily, I want a smart dispenser that manages my entire medication routine so that I can take my medicine safely and independently, while my caregiver can monitor my wellbeing remotely. Our project then aims to build a smart medicine dispenser that reminds users to take medicine, dispenses it safely, and allows basic remote monitoring through Telegram.

Sensors and Actuators used

Sensors (Inputs)	Description of use	Libraries used (if any)
DHT11	Continuously monitors ambient temperature and humidity of the storage compartment for conditions that might cause deterioration of medicine quality. Sends data over to ESP which sends an alert to Telegram informing about the temperature threshold hit.	-
Ultrasonic	Detects whether a user's hand is in front of the machine. Open the servo when detected.	-
Push Button (on MCXC)	Switch 3: Turn off buzzer Switch 2: Start heartbeat measurement	-
Heartbeat Sensor	Reads the user's pulse when they place their finger on the sensor	-

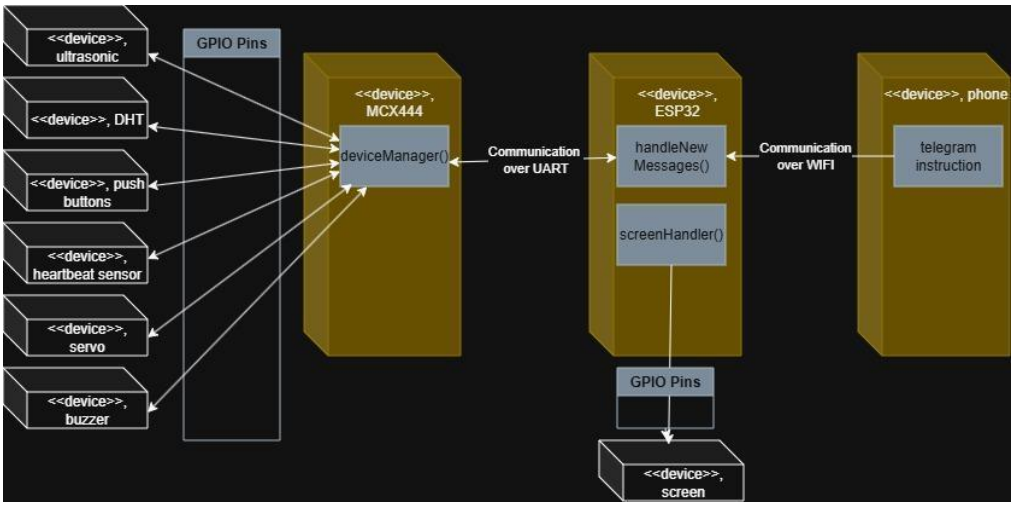
Actuators (Outputs)	Description of use	Libraries used (if any)
Servo Motor	Opens the door/flap of the medicine compartment on dispense command	-
Buzzer	Alerts the user audibly when it is time to take medicine, triggered either on a schedule or via Telegram command.	-

Screen	Displays useful information such as upcoming dispense time, heart-rate reading, and medicine status	Adafruit SSD1306 by Adafruit https://github.com/adafruit/Adafruit_SSD1306 Adafruit GFX Library by Adafruit https://github.com/adafruit/Adafruit-GFX-Library

Hardware Architecture

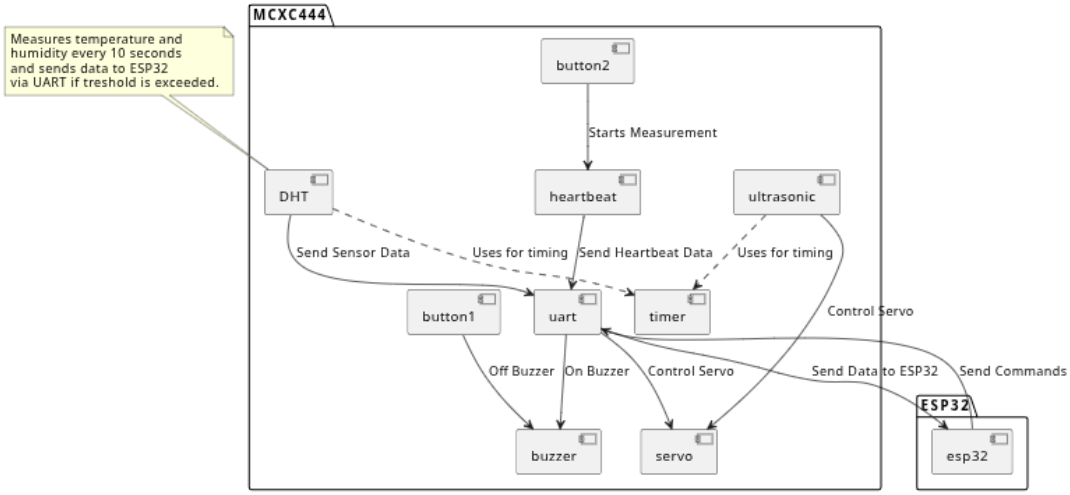
The system uses two microcontrollers working together. The MCXC444 handles local sensing, actuation, and dispensing logic, while the ESP32-S3-WROOM-1 manages the display and acts as the Wi-Fi bridge to Telegram. The two boards communicate through UART2 on the MCXC444 side (PTE22/PTE23) and Serial1 on the ESP32 side (GPIO17/GPIO18). A shared TPM0 timer running at 1 MHz provides the microsecond time base used by both the ultrasonic driver and the DHT11 bit-bang protocol. TPM1 provides the 20 ms PWM period for the servo.

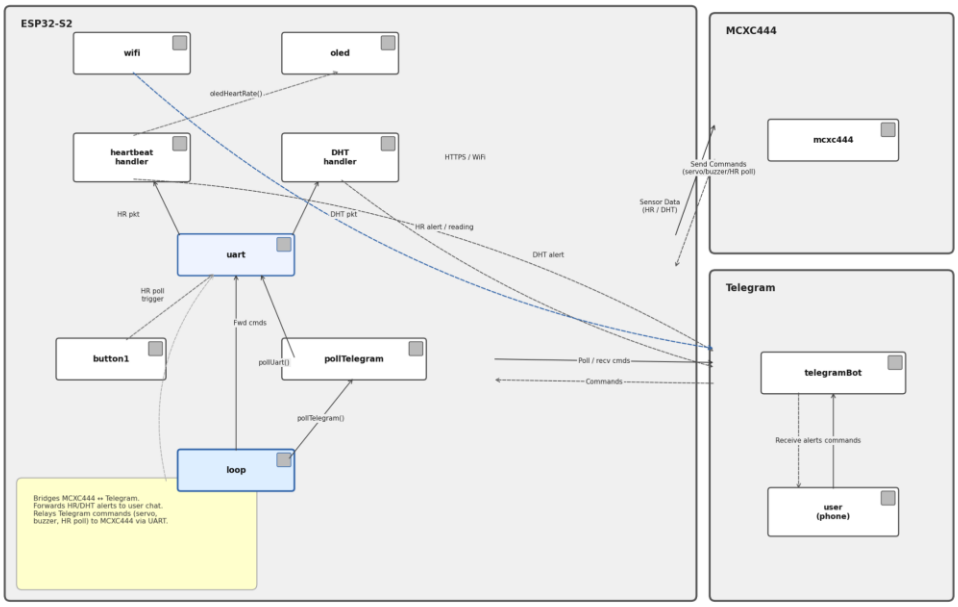
The circuit diagram below shows the connections made between the components and the devices.



Software Architecture

The dispensing sequence is driven directly by the ultrasonic task without an explicit FSM. The task maintains a local boolean servoOpen: when distance drops below 10 cm the servo is opened; when it rises above 15 cm the servo is closed. The buzzer follows a separate schedule: a FreeRTOS software timer fires every 24 hours and signals the buzzer task via a binary semaphore. A heartbeat measurement cycle is initiated on demand by pressing SW2. The software architecture diagram is shown below:





Task Structure

The table below lists all FreeRTOS tasks created on the MCXC444. Task priorities were assigned based on timing sensitivity: ultrasonic measurement is most timing-critical (priority 4), UART receive must not miss incoming data (priority 3), heartbeat should not miss any data (priority 3) and background tasks such as DHT polling and buzzer toggling run at the lowest priority (priority 1).

Task Name	Priority	Responsibility	Blocking Mechanism
ultrasonic_task	4 (highest)	Triggers ultrasonic TRIG pulse every 50 ms, then waits for echo ISR to signal pulse width via ultraSem. Computes distance; calls	xSemaphoreTake(ultraSem, 100 ms timeout)

		openServo() if < 10 cm, closeServo() if > 15 cm.	
recvTask	3	Dequeues received TPacket items from rec_queue and dispatches commands: BUZZER_DEV → buzzer_wake / buzzer_stop / buzzer_set_period_ms; SERVO_DEV → openServo / closeServo; HB_SENSOR_DEV → buzzer_wake.	xQueueReceive(rec_queue, portMAX_DELAY)
heartbeat_task	3	Waits on heartbeatSem (given by PORTC ISR on SW2 press). Runs a 10-second ADC sampling loop using adcSem to synchronise with ADC0 IRQ completions. Computes BPM, briefly sounds the buzzer, then sends a TPacket with BPM over UART.	xSemaphoreTake(heartbeatSem, portMAX_DELAY); xSemaphoreTake(adcSem, 20 ms) per sample
button_task	2	Waits on buttonSem (given by PORTA ISR on SW3 press) and calls buzzer_stop() to stop the alert.	xSemaphoreTake(buttonSem, portMAX_DELAY)
sendTask	2	Dequeues outbound TPacket items from send_queue, sets the magic byte, and writes them to UART2 by enabling the TIE interrupt. Waits on txDoneSem before sending the next packet.	xQueueReceive(send_queue, portMAX_DELAY); xSemaphoreTake(txDoneSem, portMAX_DELAY)
DHT_task	1 (lowest)	Polls DHT11 every 10 seconds. Uses taskENTER_CRITICAL for the timing-sensitive bit-bang read. Tracks consecutive high-temperature readings; sends a DHT alarm TPacket over UART after 3 consecutive readings above defined threshold.	vTaskDelay(10 000 ms); taskENTER_CRITICAL during bit-bang
buzzer_task	1 (lowest)	Waits on buzzerSem (given by the 24-hour FreeRTOS software timer callback, by recvTask on BUZZER_ON, or by heartbeatTask). Toggles buzzer GPIO every 200 ms until buzzer_stop() clears the buzzerActive flag.	xSemaphoreTake(buzzerSem, portMAX_DELAY); vTaskDelay(200 ms) per toggle

Coordination & Synchronisation

All inter-task and ISR-to-task communication uses FreeRTOS primitives. No blocking calls are made from ISR context; all ISRs use the FromISR variants and call portYIELD_FROM_ISR to yield if a higher-priority task was woken.

Primitive	Name	Producer	Consumer
Binary Semaphore	buzzerSem	buzzerTimerCallback (FreeRTOS timer), recvTask (BUZZER_ON), heartbeatTask	buzzerTask
Binary Semaphore	buttonSem	PORTA_IRQHandler (SW3 press) via xSemaphoreGiveFromISR	buttonTask
Binary Semaphore	ultraSem	Ultrasonic_PORTC_IRQHandler (echo falling edge) via xSemaphoreGiveFromISR	ultrasonic_task
Binary Semaphore	heartbeatSem	Button_PORTC_ISR (SW2 press) via xSemaphoreGiveFromISR	heartbeatTask
Binary Semaphore	adcSem	Heartbeat_ADC0_IRQHandler (conversion complete) via xSemaphoreGiveFromISR	heartbeatTask (per ADC sample)
Binary Semaphore	txDoneSem	UART2_FLEXIO_IRQHandler (TX complete) via xSemaphoreGiveFromISR	sendTask (serialises back-to-back transmissions)
Queue (depth 5)	send_queue	uart_send() called by DHTTask, heartbeatTask	sendTask
Queue (depth 5)	rec_queue	UART2_FLEXIO_IRQHandler (full packet received) via xQueueSendFromISR	recvTask

ISR design summary:

ISR	Trigger	NVIC Priority	Action
PORTA_IRQHandler	PTA4 falling edge (SW3)	192 (lowest)	Clears flag; gives buttonSem via xSemaphoreGiveFromISR

PORTC_PORTD_IRQHandler	PTC1 rising/falling edge (ultrasonic echo); PTC3 falling edge (SW2)	128	Records echo timestamps in TPM0->CNT; gives ultraSem on falling edge; calls Heartbeat_StartMeasurement on SW2
UART2_FLEXIO_IRQHandler	UART2 TDRE or RDRF	128	TX: sends next byte or disables TIE and gives txDoneSem. RX: buffers bytes; on magic byte match assembles full TPacket and gives to rec_queue via xQueueSendFromISR
ADC0_IRQHandler	ADC0 conversion complete	192 (lowest)	Reads ADC0->R[0]; passes value to Heartbeat_ADC0_IRQHandler which gives adcSem via xSemaphoreGiveFromISR

Resource Protection

The UART2 peripheral is protected against concurrent access by txDoneSem: sendTask holds the semaphore for the entire duration of a transmission (from enabling TIE to the TDRE interrupt signalling completion), so no second packet can begin transmitting until the first is fully sent. The DHT11 bit-bang read wraps the timing-sensitive section in taskENTER_CRITICAL / taskEXIT_CRITICAL to prevent scheduler preemption from corrupting the microsecond-level timing required by the single-wire protocol.

Communication Protocol

UART Configuration

Both devices communicate over a UART link configured as follows:

- Baud rate: 9600
- Frame: 8 data bits, no parity, 1 stop bit (8N1)
- Hardware flow control: none
- MCXC444: UART2, TX on PTE22, RX on PTE23, NVIC priority 128
- ESP32-S3: Serial1, TX on GPIO17, RX on GPIO18

Packet description

All messages between the two boards use a fixed-size TPacket structure (21 bytes total) defined in constant.h on the MCXC444 side. The magic byte (0x67) is used by the UART2 ISR receiver to detect the start of a new packet and re-synchronise after noise or partial frames.

Field	Size	Description	Example Values
magic	1 byte	Framing byte used to detect the start of a valid packet	0x67
device_type	1 byte	Identifies the device originating or targeted by the packet	0=BUZZER_DEV, 1=BUTTON_DEV, 2=HB_SENSOR_DEV, 3=HDT_SENSOR_DEV, 4=SERVO_DEV
command	1 byte	Action to perform or event being reported	0=SERVO_OPEN, 1=SERVO_CLOSE, 2=BUZZER_ON, 3=BUZZER_OFF, 4=BUZZER_CHANGE, 5=HB_GET_DATA, 6=CMD_NONE
padding[2]	2 bytes	Word-alignment padding (reserved, set to 0x00)	0x00 0x00
data[16]	16 bytes	Payload. For DHT alarm: data[0]=temperature_c, data[1]=humidity_percent. For heartbeat: data[0]=BPM. For BUZZER_CHANGE: data[0]=hours.reading or status string	e.g. {40, 85} for DHT alarm; {72} for 72 BPM

Additional Features

The UART link is bidirectional:

- MCXC444 → ESP32: DHTTask sends a TPacket (device=DHT_SENSOR_DEV, data[0]=temperature, data[1]=humidity) when a high-temperature alarm fires. heartbeatTask sends a TPacket (device=HB_SENSOR_DEV, data[0]=BPM) after each measurement. The ESP32 formats these and forwards them to the Telegram chat.
- ESP32 → MCXC444: Telegram commands from the caregiver are translated into TPackets by the ESP32 and sent over UART. recvTask on the MCXC444 dispatches them to buzzer or servo functions accordingly. BUZZER_CHANGE additionally carries the new interval in hours in data[0].

Online Function Used

The ESP32-S3 connects to the Telegram Bot API over HTTPS using the UniversalTelegramBot library, which makes up the internet/cloud component. The bot acts as both a sender (pushing sensor data and device status to the caregiver's Telegram chat) and a receiver (accepting remote commands from the caregiver).

Libraries used:

- UniversalTelegramBot by Brian Lough — https://github.com/witnessmenow/Universal-Arduino-Telegram-Bot
- ArduinoJson by Benoit Blanchon — https://github.com/bblanchon/ArduinoJson
- WiFiClientSecure (built-in ESP32 Arduino core)
- Adafruit SSD1306 by Adafruit — https://github.com/adafruit/Adafruit_SSD1306
- Adafruit GFX Library by Adafruit — https://github.com/adafruit/Adafruit-GFX-Library

The following Telegram commands are supported:

Command	Action on ESP32	UART Packet sent to MCXC444
/buzzer_on	Triggers a buzzer alert remotely	device=BUZZER_DEV, cmd=BUZZER_ON
/buzzer_off	Silences the buzzer remotely	device=BUZZER_DEV, cmd=BUZZER_OFF
/servo_open	Triggers a remote dispense	device=SERVO_DEV, cmd=SERVO_OPEN
/servo_close	Closes the medicine compartment flap	device=SERVO_DEV, cmd=SERVO_CLOSE
/status	Returns ESP32 free heap and uptime. No UART packet sent.	-
/heartrate	Polls the heart rate sensor	device=HB_SENSOR_DEV, cmd=HB_GET_DATA
/buzzer_change<1-24>	Sets the buzzer reminder interval in hours	device=BUZZER_DEV, cmd=BUZZER_CHANGE, data[0]= hours
/start, /help	Lists all available commands	-

Data sent by ESP32 to Telegram (MCXC444 → ESP32 → Telegram): when the MCXC444 sends a TPacket over UART, the ESP32 formats and forwards it to the caregiver's chat. The message includes the device name, command name, and any data payload (e.g. a sensor reading string).

Sample of Telegram messages

Attached below is an image of the Telegram bot working. Buzzer on/off commands are acknowledged instantly, and two /heartrate polls return live sensor readings

Challenges faced & Solutions

UART Communication Between MCXC444 and ESP32

Initial testing showed no data being received on either side despite seemingly correct software configuration. The root cause was traced to two hardware issues: a missing shared ground between the two boards, and a voltage level mismatch — the MCXC444 operates at 3.3 V while the ESP32's UART lines also nominally operate at 3.3 V but were floating without a common reference. Once a shared GND wire was added, communication became stable.

Packet Framing and Receiver Synchronisation

Early in development, the UART receiver would occasionally misalign on the byte stream, causing packets to be parsed with incorrect offsets and producing garbage data. The solution would be to introduce the magic byte (0x67) framing scheme in the Tpacket protocol, combined with a timeout-based reset in the FSM of the ESP32. If the gap exceeded 100ms, the receiver would discard the partial buffer and resynchronise on the next magic byte

Conclusion

This project successfully demonstrated the design and implementation of a smart medicine dispenser using two cooperating microcontrollers — the MCXC444 and the ESP32-S2 — coordinated through a reliable UART communication protocol. The system achieves its core goal of supporting elderly medication management by combining scheduled buzzer reminders, proximity-triggered dispensing via the ultrasonic sensor, real-time heart rate monitoring, and remote caregiver access through a Telegram bot.

The use of FreeRTOS on the MCXC444 allowed concurrent management of time-sensitive tasks such as ultrasonic echo measurement and UART reception alongside lower-priority background tasks like DHT11 polling and buzzer scheduling, with all inter-task communication handled safely through semaphores and queues. The bidirectional TPacket protocol provided a robust and extensible communication layer between the two boards, enabling both remote command delivery and autonomous sensor alert forwarding.